

Avani Tiwari
Jonathan Guan
Jeffrey Rathgeber

Applied Cryptography Project 1 Report

Cryptanalysis of a class of ciphers based on a variation of the poly-alphabetic substitution cipher (including insertion of random characters), decrypted by first using an Index of Coincidence (IOC) algorithm to determine the length of the key, a comparison to an English letter frequency table to find the most likely key, then unshifting the ciphertext, and selecting the most likely matching plaintext.

One cryptanalysis approach is being submitted by Avani Tiwari, Jonathan Guan, and Jeffrey Rathgeber. After multiple brainstorming meetings, one cohesive approach was selected to implement, as described above. Multiple different approaches were considered, such as: variations of a longest common substring algorithm to determine the key length, comparison to English letter frequency tables to determine key length, as well as an exhaustive search algorithm. After considering the efficiency and consistency of different algorithms, we decided to use different aspects of each potential approach to arrive at the algorithm that was put together. The majority of the original pseudo code was the brainchild of Jonathan, and he implemented the algorithms using the English letter frequency table to determine the key, as well as the algorithm to arrive at the plaintext once the key was cracked. Avani implemented the first part of the decryption algorithm, which used the index of coincidence to arrive at the most probable key length with a very high success rate, assuming the key does not have an unnatural amount of repeated letters. Jeffrey implemented an encryption algorithm in such a way to mimic the encryption process of the given pseudo code. All three team members brought valuable input to the brainstorming process, and the bouncing of ideas off of one another is what allowed us to arrive at one harmonious solution using different aspects of each different potential approach.

Our implementation can be intuitively divided into 3 main parts, each with their own inputs (taken from previous part) & outputs (to be fed into next part):

Part I

input(s): encrypted ciphertext of length = length of plaintext + number of random characters

output(s): resolved key length, list of slices of ciphertext of size = key length

Methodology:

We begin with slicing the strings at interval t which goes from 1- max possible length. We create a list with max length lists in it. The slices of various lengths get added to the specific list. Then we calculate the index of coincidence for each of the slices and average it for each key length. The index of coincidence value is high for the multiples of the actual key length. In order to get the exact one we calculate the factors of the key_length we have got and compare the average index of coincidence value for all the factors to a threshold value which is 1.7 for the English language (1). On finding the most probable key length we return it and the list of slices for that key length that is later used to make the frequency table.

Index of coincidence is a function that tells the likelihood of occurrence of a character in the slices. The formula for index of coincidence is -

$I = 27 * \sum n_i (n_i - 1) / N (N - 1)$ where n_i is the number of times the character occurs in the text and N is the total size of the text

Pseudocode ->

- *index_of_coincidence* function-
 - Input - string text
 - Initialize letter to a string with all the english alphabets and a space
 - Create list count to store count of each letter
 - Initialize the list count with 0s for each letter
 - Initialize numerator to 0
 - Initialize letter_count to 0
 - For each character in string text
 - Increment the count for the characters in the string text in the list count
 - For each value in range (0, 27)
 - numerator = numerator + count[i]*(count[i]-1)
 - letter_count = letter_count +count[i]
 - Return $27 * \text{num} / (\text{letter_count} * (\text{letter_count}-1))$
- *key_length* function -
 - Input- text, the maximum key length possible
 - Create slices list which is a list containing max_key_length +1 number of lists
 - Create quotient_dictionary which is a dictionary that stores the value we get on dividing the length of the text with the length we are slicing for - this gives the the number of elements that number of slices we get for that particular key length
 - For each element in range (1, max_key_length+1)
 - Initialize quotient as the value we get on diving the text length by the current_key_length
 - If q is in quotient_dictionary

- Continue
- Else
 - Add the quotient to the quotient_dictionary
- In the list for the current_key_length in the list slices we add current_key_length number of empty strings
- For slice_number in range current_key_length
 - Initialize a variable l as the current_slice_number
 - While l < length of the text inputted
 - slices[current_key_length][current_slice_number] += text[l]
 - l += current_key_length
- Create ioc list of lists that stores the index of coincidence value for each slice for all the key lengths
- for all the key_lengths in the range(length of slices)
 - initialize the list ioc[current_length] with same number of 0s as the number of slices for the current_key_length
 - for each slice in slices[current_key_length]:
 - ioc[current_key_length][current_slicenumber] = index_of_coincidence(slices[current_key_length][current_slicenumber])
- Create a list called average_ioc that stores the average ioc values for the given key_length
- for each key_length in ioc
 - Append the average of iocs of the slices to the list average_ioc
- Initialize the variable final_key_length that stores the index of the maximum value in the average_ioc list +1
- for each value in range (1, final_key_length +1)
 - if the key_length % current_value ==0
 - if the average_ioc[current_value] > 1.7
 - final_key_length = current_value
 - break
- return final_key_length, slices[final_key_length]

Part II

input(s): resolved key length, list of slices of ciphertext, English letter frequency table

output(s): list of integers representing by how much each character was shifted by in each ciphertext slice

Methodology:

The goal of part 2 is to determine the key from the ciphertext, given the length of the key as determined in part 1. To do this, we will examine the distribution of letter frequency found

within each slice of the original ciphertext. Assuming the key length provided is accurate, the letters within each slice should be encoded using the same character shift, effectively becoming a mono-alphabetical cipher. To determine the correct shift at each slice, we first build a frequency table, which holds the frequency that each character appears (from a to z, and blank space) in order. The distribution of letter frequency should match up somewhat closely to the distribution of average english letter frequencies. Therefore, we can rotate the frequency table 27 times, and on each rotation, compare the frequency of the table to the english letter frequency (matching all letters from a-z, ignoring blank space characters). We compute the error by summing the differences in the frequency tables. The rotation with the best match (least error) to the english letter frequency table will be the guess for the shift at that position in the key. By repeating this step in all slices, we derive the best guess for the encryption key.

Input: key_length = size of the key, slices = array of strings, where each string holds a subset of every character exactly key_length apart in the ciphertext

Pseudocode ->

- Result = [] //stores final key
- Slice_distr = [] //stores letter distribution of each slice
- For slice in slices:
 - For letter in the alphabet
 - Slice_distr[slice_index][index of letter] = avg occurrence of [letter] in slice //index of letter = 0-25 corresponding to a-z
 - Slice_distr[slice_index][26] = avg occurrence of [' '] in slice
- English_freq_table = imported frequency table
-
- For distr in slice_distr:
 - errors[]
 - Loop i = 27 times:
 - Error = 0
 - Loop j = 26 times:
 - Error += abs(distr[j] - english_freq_table[j]) // sum error between tables
 - Errors[i] = error
 - Rotate distr
 - Insert (indexOf(min error(errors))) into result
- Return result

Part III

input(s): original undivided ciphertext, list of integers representing by how much each character was shifted by in each ciphertext slice

output(s): decrypted ciphertext after removing random characters (correct choice of 5 potential options for test 1, or correct ordering of substrings for test 2)

Methodology:

Part 3 is involved with deriving the correct plaintext using the key found in part 2. It attempts this by first decrypting the ciphertext using the key provided. This results in a half decoded “plaintext”, except for the random characters inserted throughout the text. This decoded text is then compared first to the texts from dictionary 1, using the levenshtein distance formula, which finds the number of inserts or deletions required to go from one string to another. Using this formula allows us to get a low error when the text mostly matches, only offset due to the random insert characters, whereas comparing with texts that don’t match at all will return much larger errors. If no texts from dictionary 1 match with an error less than a certain threshold, then we proceed to matching dictionary 2. The half-decoded message is then split into an array of words. For each word, the algorithm searches through every word in dictionary 2, and calculates an error using levenshtein. The word with the least error is selected, and added to the result plaintext. After iterating through every word, the result plaintext is returned.

Pseudocode ->

Input: key = array containing number of shifts at each character, ciphertext = encoded message

- Dict1 = [] //array holding plaintexts from dictionary 1
 - Read from dict1.txt
- Dict2 = [] //array holding words from dictionary 2
 - Read from dict2.txt
- decoded_message = “”
- Loop with i: //decode the message using key
 - intchar = 0-25 if ciphertext[i] is a-z, and 26 if ciphertext[i] is ‘ ‘
 - intchar -= key[i % keylength] // shift the character using key
 - if (intchar < 0) intchar += 27 // loop back around if rotated past ‘a’
 - decoded_message += char(intchar + 97)
 - except if(intchar = 26), decoded_message += ‘ ‘ //add shifted char to the decoded message
- For plaintext in dict1:
 - error = levenshtein_distance(plaintext, decoded_message)/len(decoded_message)
 - if (error < 0.15), return plaintext
- Separated_decoded_message = [] //array holding decoded_message split by blank space(each word)
- Result_message = “”

- For each word in decoded_message:
 - Max_error = infinite
 - Best_word = ""
 - For each dict_word in dict2:
 - If ((error = levenshtein_distance(word, dict_word)) < max_error)
 - Best_word = dict_word, max_error = error
 - Result_message += best_word + " "
- Return result_message

Works Cited

1. *Five ways to crack a Vigenère cipher - national cipher challenge 2022*. (n.d.). Retrieved October 12, 2022, from <https://www.cipherchallenge.org/wp-content/uploads/2020/12/Five-ways-to-crack-a-Vigenere-cipher.pdf>